

Gestión de datos Streaming

Índice

1. Entornos distribuidos para arquitecturas Streaming
 - 1.1 Contexto
 - 1.2 Arquitecturas Streaming
2. Arquitectura de Apache Kafka
 - 2.1 Introducción
 - 2.2 Conceptos
3. Herramientas y uso de Apache Kafka
 - 3.1 Ecosistema
 - 3.2 Plataforma Confluent

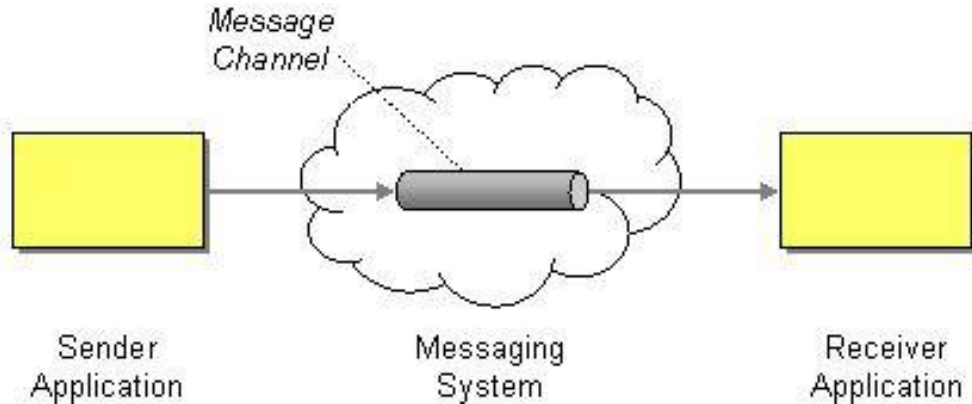
Entornos distribuidos para arquitecturas Streaming

Contexto

- Problemas de la integración de aplicaciones:
 - Red: poco confiable y lenta
 - Tecnologías heterogéneas
 - Cambios y evolución
- 4 aproximaciones:
 - Intercambio de ficheros
 - Base de datos
 - Remote Procedure Invocation
 - Mensajes

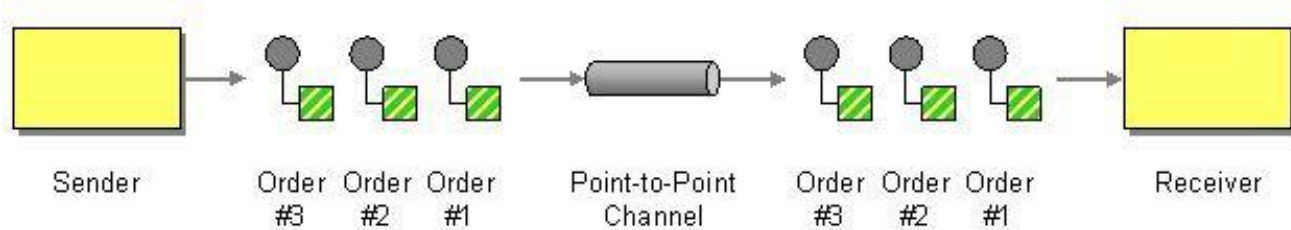
Contexto

- Mensajería
 - Tecnología que permite comunicación entre aplicaciones:
 - Alta velocidad
 - Asíncrona
 - Entrega fiable



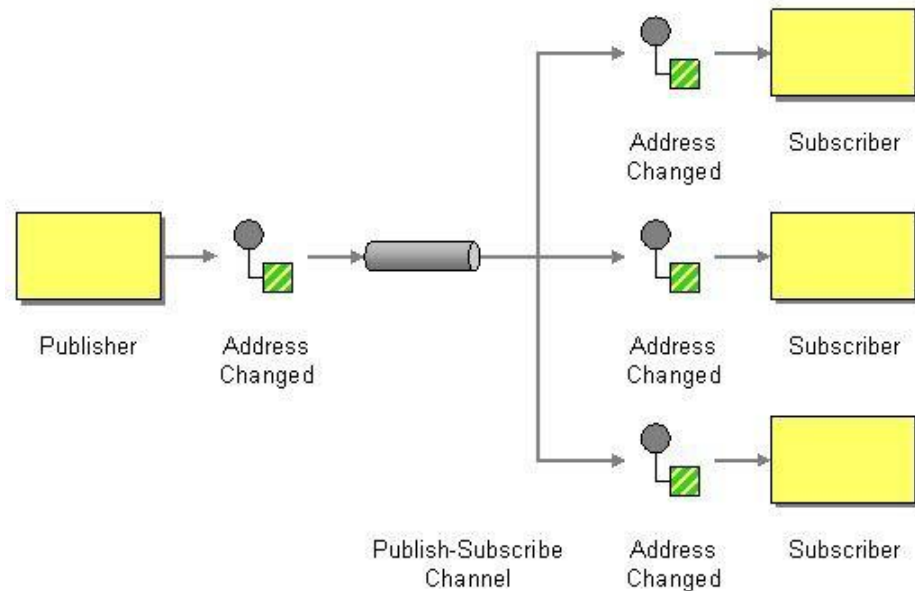
Contexto

- Canal punto a punto
 - Concepto de cola de mensajes
 - Si hay varios receptores, solo uno de ellos puede consumir cada mensaje



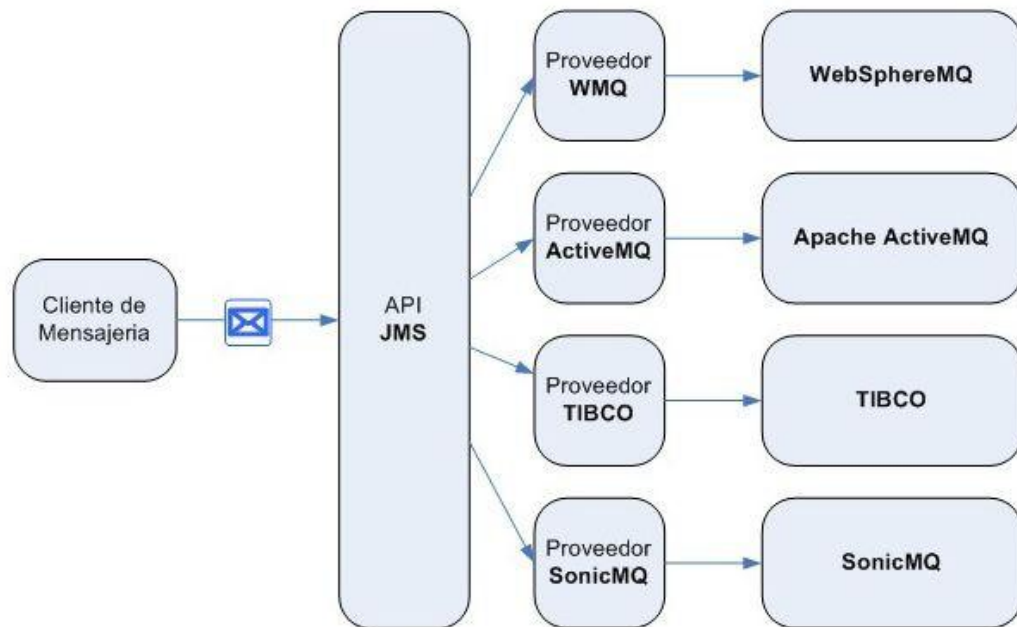
Contexto

- Canal Publicación-Subscripción (Publish-Subscribe)
 - Concepto de tópico
 - Cada suscriptor recibe una copia de cada mensajes



Contexto

- Java Message Service



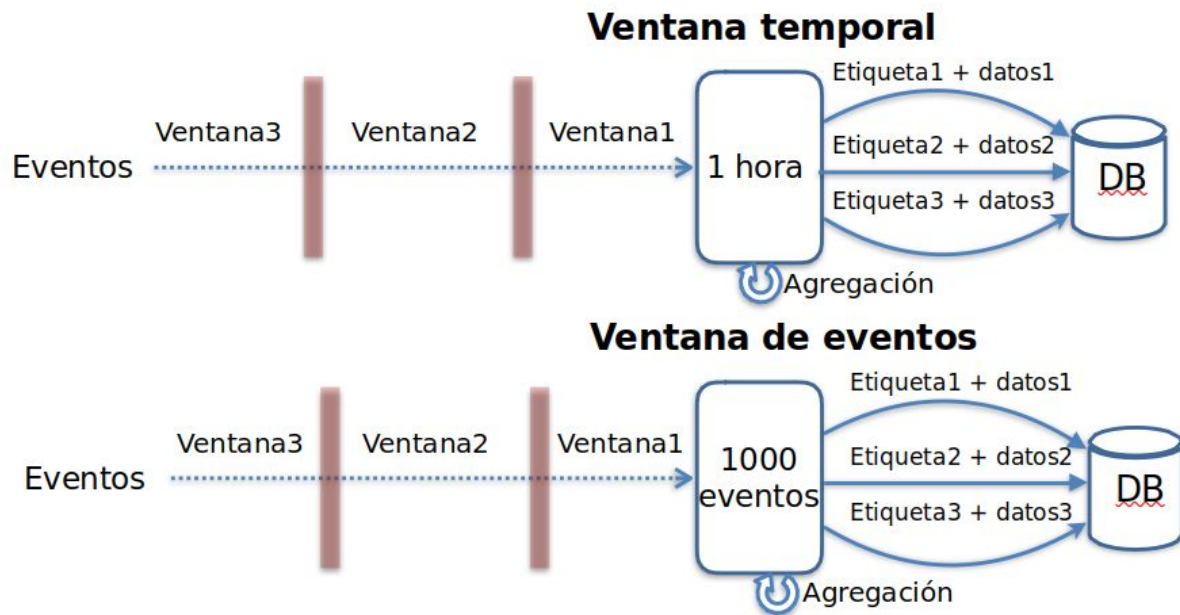
Contexto

- Otros protocolos. Alternativas más modernas:
 - AMQP
 - RabbitMQ (colas, exchange, bindings, routing key)
 - MQTT (Message Queue Telemetry Transport)
 - Protocolo ligero usado para IoT, sensores, etc.

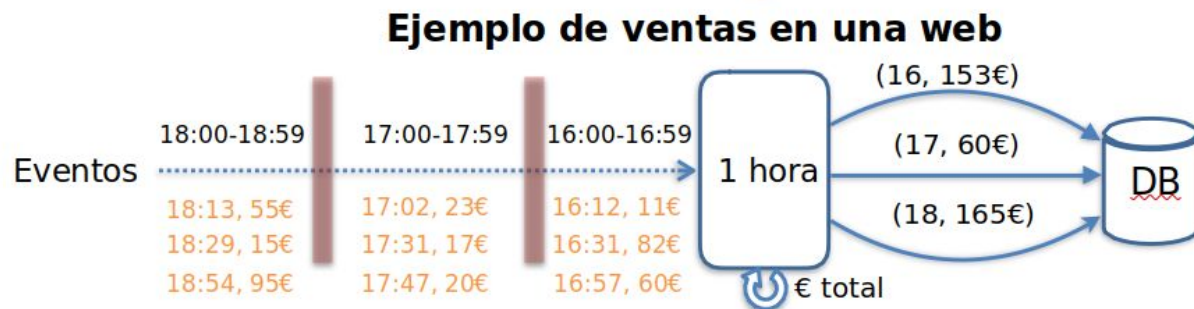
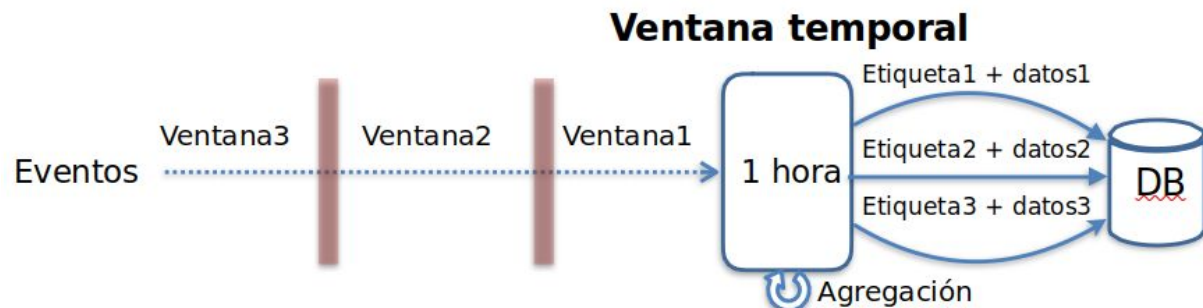
Arquitecturas Streaming

- Características:
 - El sistema que queremos monitorizar equivale a una corriente continua e infinita de eventos o de datos
 - El flujo tiene un inicio pero no un final
 - No pueden tratarse todos los eventos a la vez, como en batch, ya que son potencialmente infinitos. Deben usarse ventanas temporales o por número de eventos
 - Respuesta (casi) inmediata
 - Algunos casos de uso típicos: IoT, fraude, ciberseguridad, dashboard en tiempo real, monitorización en tiempo real, etc.
 - Tecnologías: Flume, Kafka, Spark Streaming, Storm, Kinesis, etc.

Arquitecturas Streaming

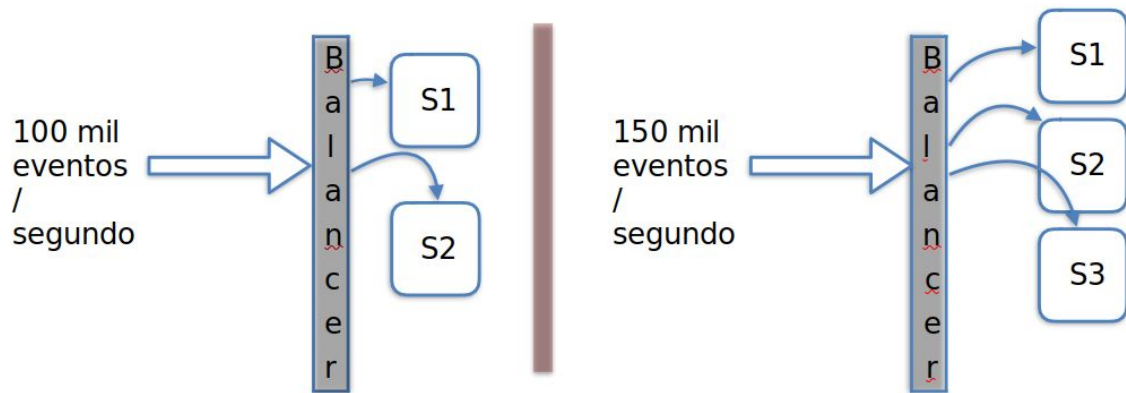


Arquitecturas Streaming



Arquitecturas Streaming

- Escalabilidad:
 - Capacidad de un sistema para tratar mayor cantidad de datos cuanto más recursos se añaden al cluster



Arquitecturas Streaming

- Backpressure
 - Proceso Streaming: un emisor envía datos a un receptor de forma continua
 - El emisor puede emitir eventos a una mayor velocidad a lo que el receptor puede procesar, provocándole una saturación o pérdida de datos
 - Solución: el receptor le informa al emisor de su capacidad para que reduzca el ritmo de emisión



Ritmo:

1000 eventos / seg

Capacidad:

500 eventos / seg

Arquitectura de Apache Kafka

Introducción a Apache Kafka

- <http://kafka.apache.org/>
- Creado en LinkedIn, open sourced en el año 2011
- Implementado en Scala (principalmente) y en Java
- 9 core committers, + ~ 20 contributors
- Los desarrolladores de Apache Kafka fundaron en el año 2014 la startup Confluent
- <https://www.confluent.io/blog/confluent-raises-a-125m-series-d-funding-round>
- <http://www.confluent.io/training>



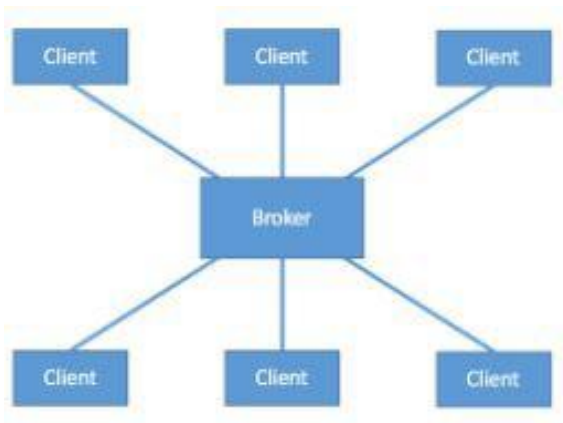
Introducción a Apache Kafka

- Motivación: plataforma unificada para el manejo de todos los datos en tiempo real de una gran empresa
- Características:
 - Alto rendimiento
 - Durabilidad
 - Alta disponibilidad
 - Escalabilidad



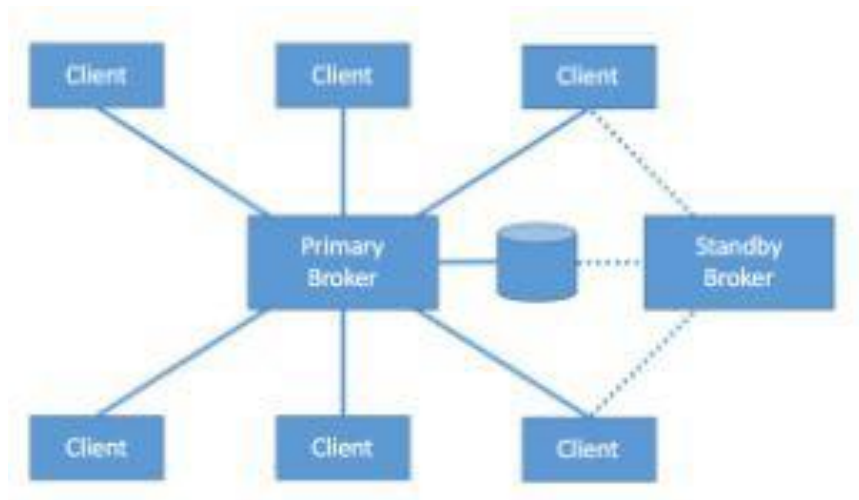
Introducción a Apache Kafka

- Sistemas tradicionales:
 - El productor envía los mensajes al broker
 - El broker los guarda (memoria o disco)
 - El cliente los consume
 - El broker borra el mensaje



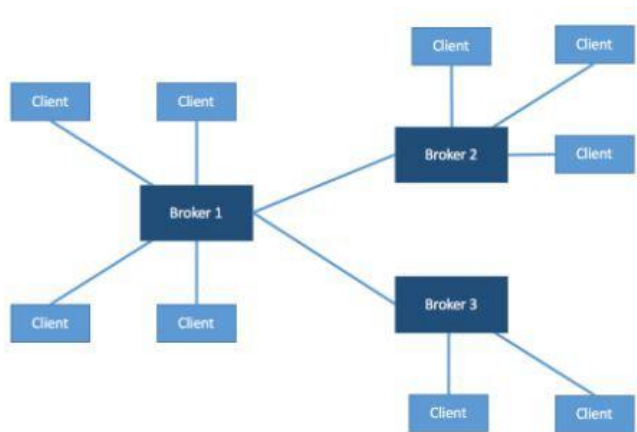
Introducción a Apache Kafka

- Sistemas tradicionales:
 - Evolución: broker en HA
 - Los clientes solo se conectan al primario (que contiene todas las colas y tópicos)



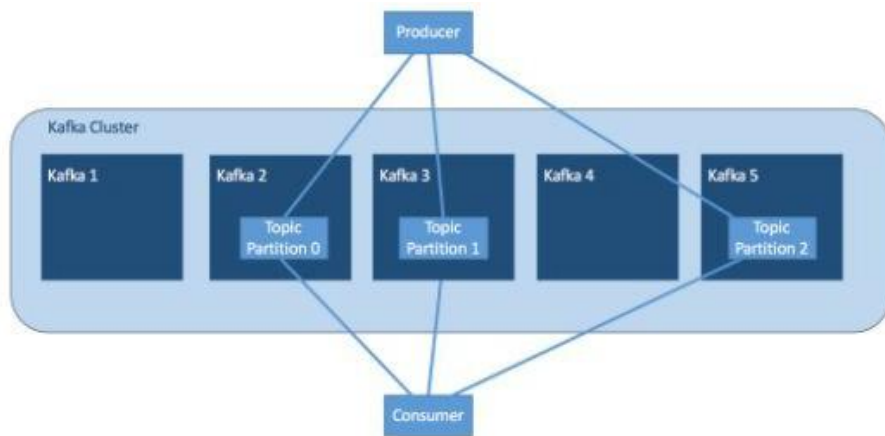
Introducción a Apache Kafka

- Sistemas tradicionales:
 - Los brokers se dividen los tópicos
 - Los clientes se conectan solamente al broker que contiene su tópico
 - No escala horizontalmente



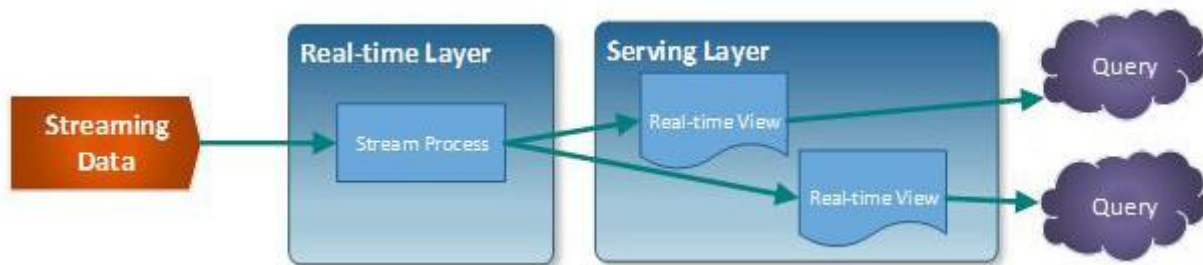
Introducción a Apache Kafka

- Propuesta de Kafka:
 - Tópico dividido en diferentes brokers y replicado
 - Escalado horizontal + vertical
 - Todos los mensajes son persistidos a disco, no se borran tras la entrega al consumidor



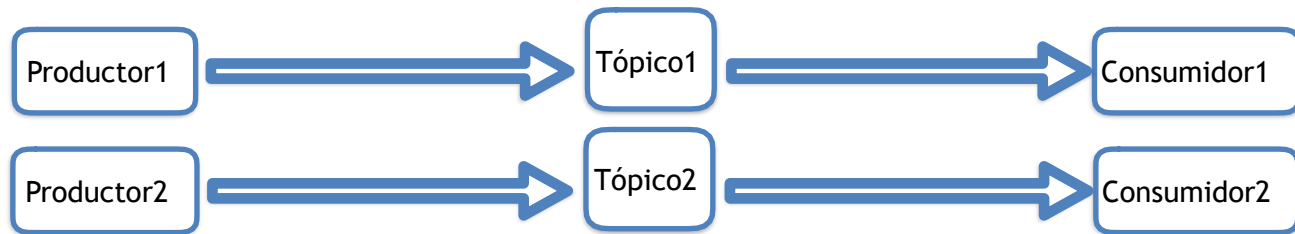
Introducción a Apache Kafka

- Arquitectura Kappa:
 - Todo es un stream
 - Los datos de partida no se modifican
 - Posibilidad de reprocesado
 - Garantía de orden en el procesamiento de mensajes



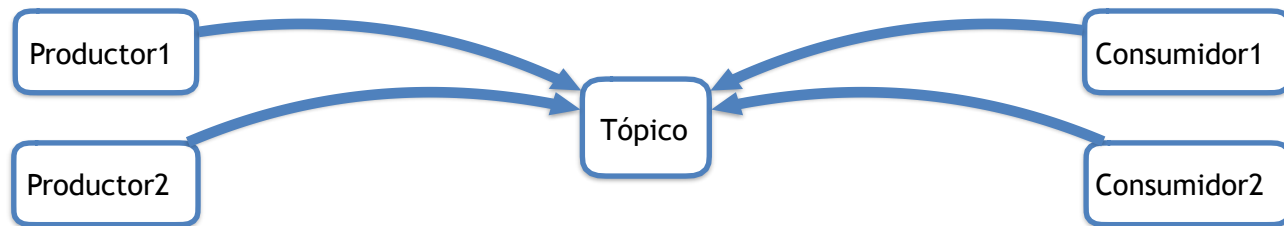
Arquitectura Apache Kafka

- Herramienta escalable de Streaming que actúa como un “buffer” de datos entre otras aplicaciones de Streaming
- Permite solventar el problema de backpressure para sistemas que no tienen soporte para ello
- Emisor = Productor. Receptor = Consumidores
- Se emiten y se consumen eventos de un “tópico”



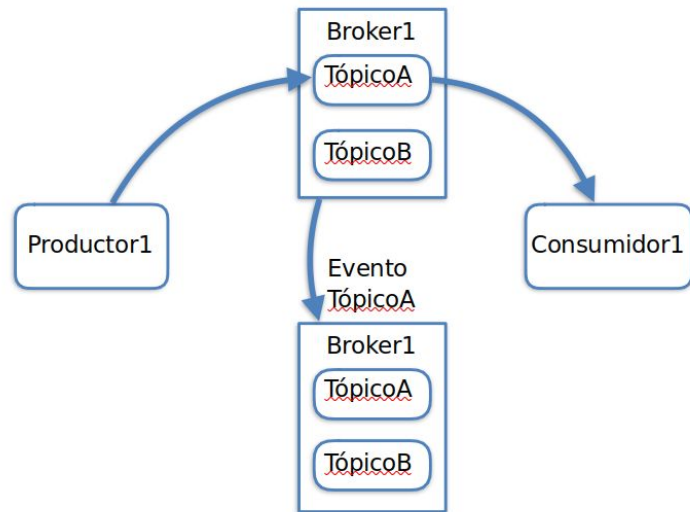
Arquitectura Apache Kafka

- Modelo Publish-Subscribe
- Un productor solo emite a un tópico
- Un consumidor solo recibe de un tópico
- Varios productores pueden emitir al mismo tópico
- Varios consumidores pueden recibir del mismo tópico



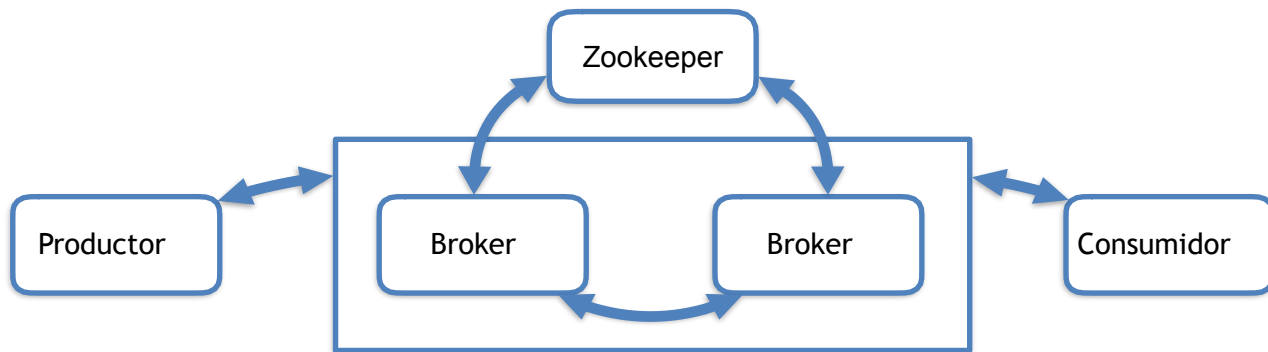
Arquitectura Apache Kafka

- Un broker es un servidor que permite almacenar eventos de uno o varios tópicos
- Una conjunto de brokers forman un cluster
- Cada broker contiene réplicas de datos = alta disponibilidad



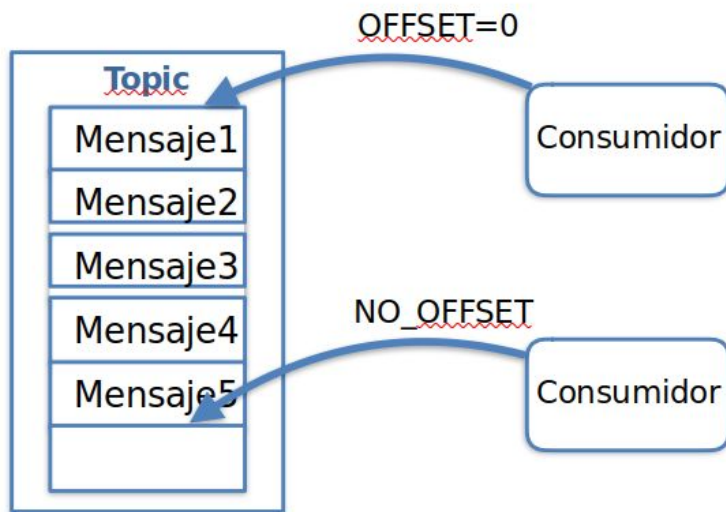
Arquitectura Apache Kafka

- Los productores, los brokers y los consumidores coordinan sus procesos a través de Zookeeper
- Los productores y los consumidores utilizan ZK para saber dónde están los brokers



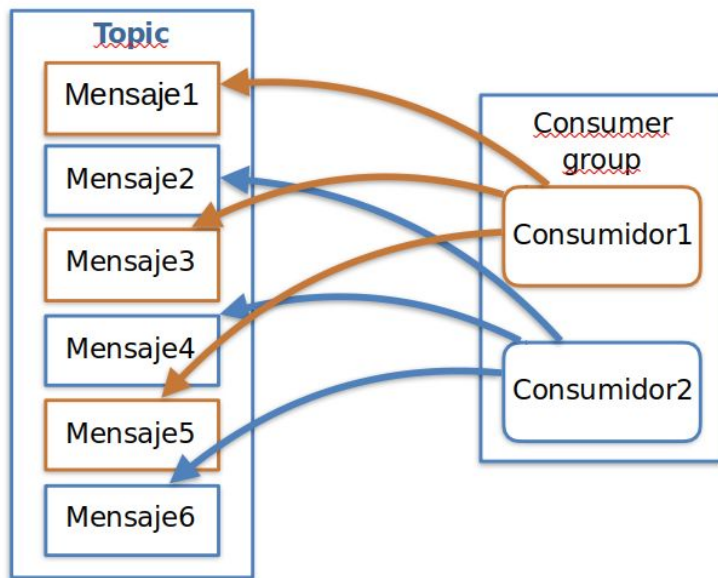
Arquitectura Apache Kafka

- Kafka guarda todos los eventos de forma permanente (normalmente entre 2 y 7 días)
- Se puede acceder a cualquier evento a través del offset (desplazamiento)
- Por defecto, se usa el offset actual

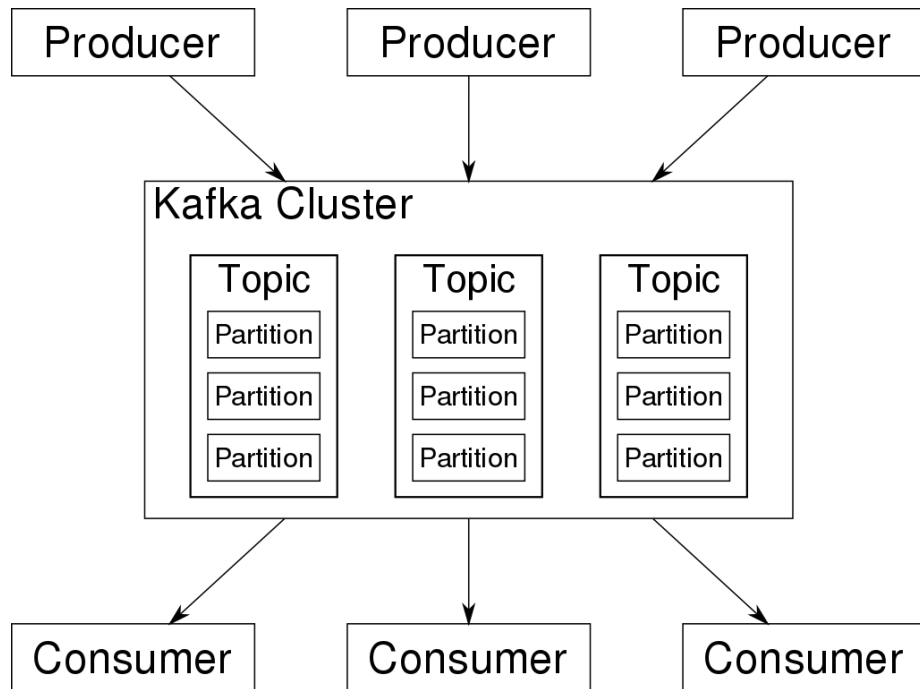


Arquitectura Apache Kafka

- Varios consumidores pueden compartir el offset sobre un topic, formando así un consumer group
- Cada consumidor recibe un mensaje del tópico sin que se repita entre consumidores

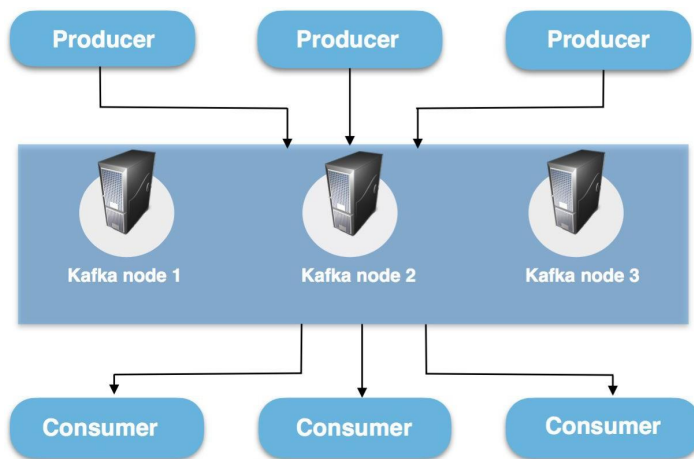


Conceptos de Apache Kafka



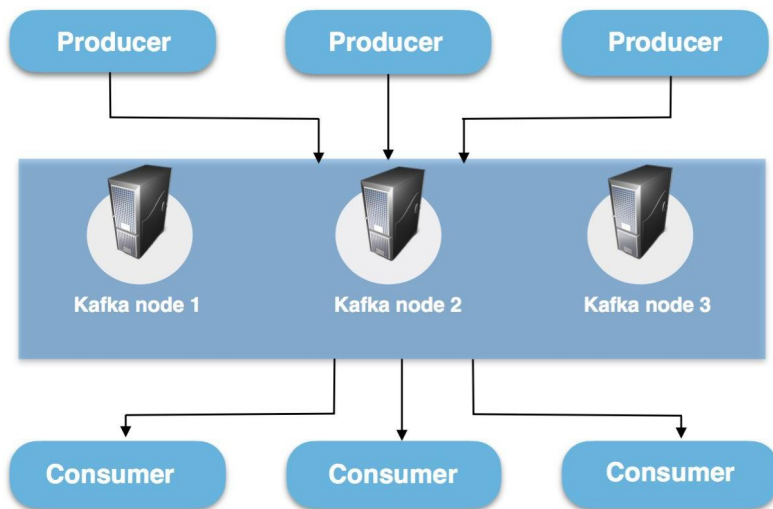
Conceptos de Apache Kafka

- Brokers
 - 1 o más brokers
 - Todos con broker.id único



Conceptos de Apache Kafka

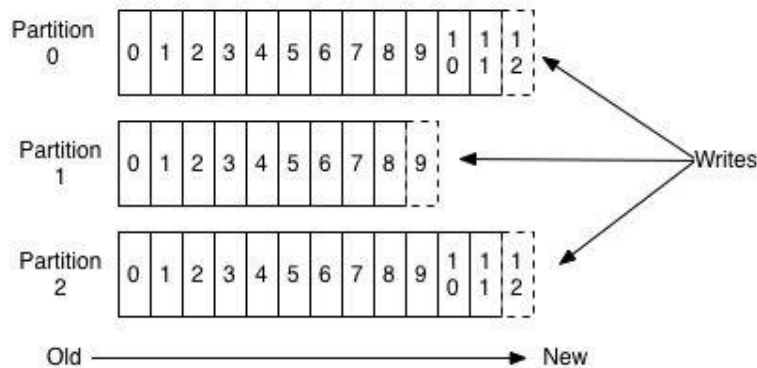
- Brokers
 - 1 o más brokers
 - Todos con broker.id único



Conceptos de Apache Kafka

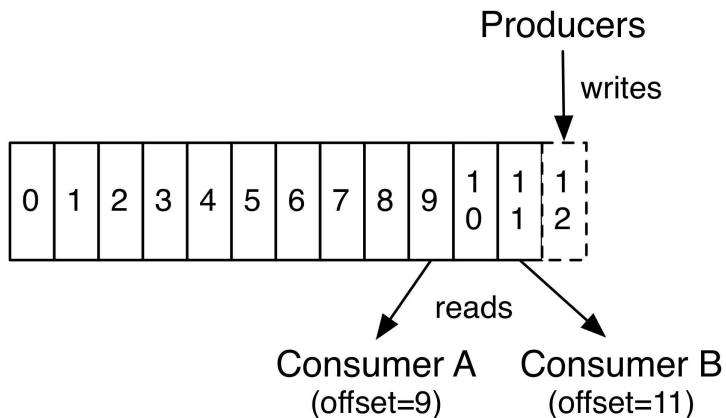
- Tópicos
 - Nombre o destino al que se publican los mensajes
 - Pueden tener 0,1, o varios consumidores
 - Se dividen en 1 o varias particiones

Anatomy of a Topic



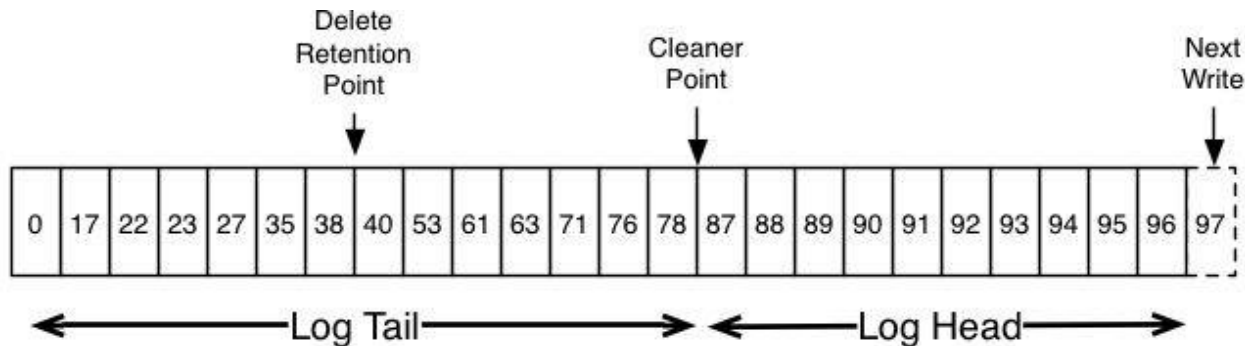
Conceptos de Apache Kafka

- Particiones
 - Secuencia ordenada e inmutable de registros
 - Append only (log)
 - Offset secuencial
 - Unidad de paralelismo y escalado
 - Se pueden replicar



Conceptos de Apache Kafka

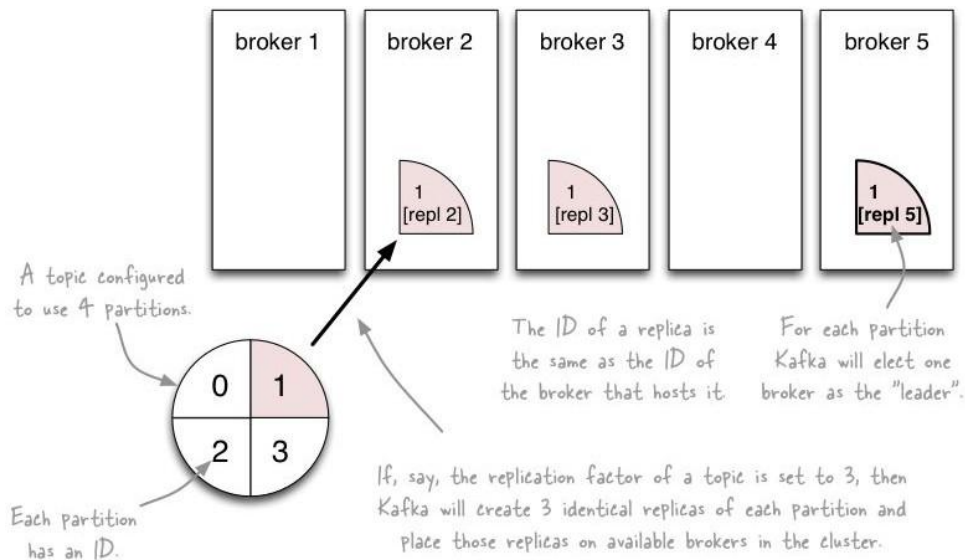
- Records
 - Se identifican unívocamente por topic, partición y offset
 - Par clave/valor. Todos los mensajes con la misma clave se envían a la misma partición
 - Se guardan en disco (sean consumidos o no) hasta cumplir su retention policy



Conceptos de Apache Kafka

- Particiones y réplicas:

<http://www.michael-noll.com/blog/2013/03/13/running-a-multi-broker-apache-kafka-cluster-on-a-single-node/>



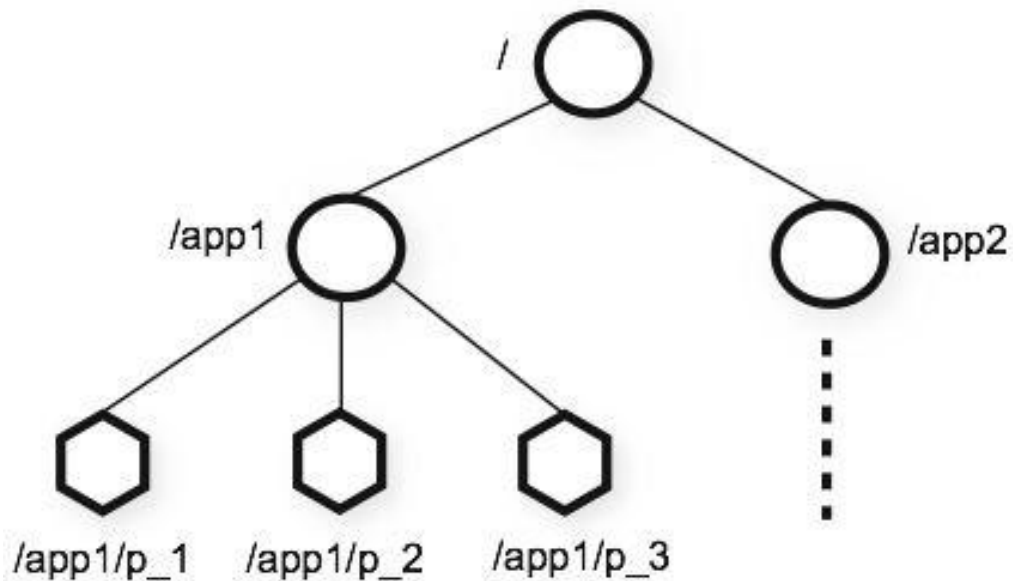
Conceptos de Apache Kafka

- Apache Zookeeper
 - Sistema open source de coordinación para aplicaciones distribuidas
 - Casos de uso:
 - Gestión de configuración
 - Sincronización distribuida
 - Elección de líder
 - DNS
 - Registro de datos centralizado



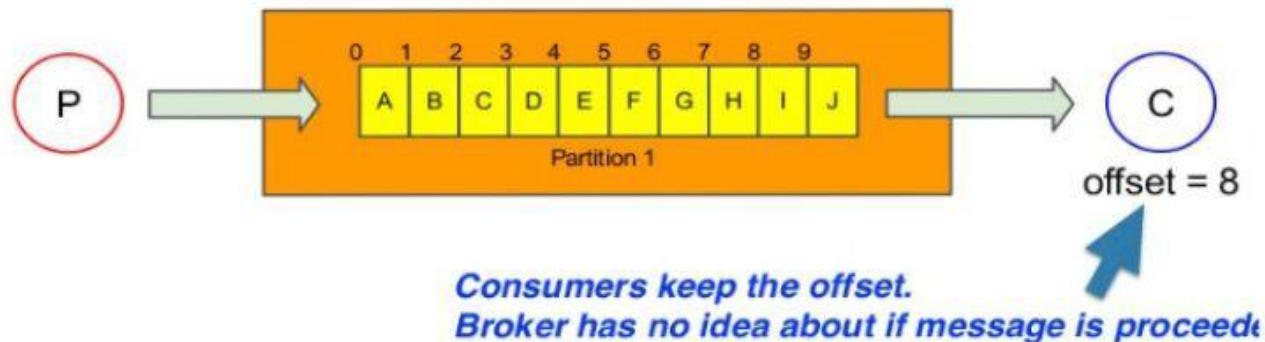
Conceptos de Apache Kafka

- Apache Zookeeper
 - znodes



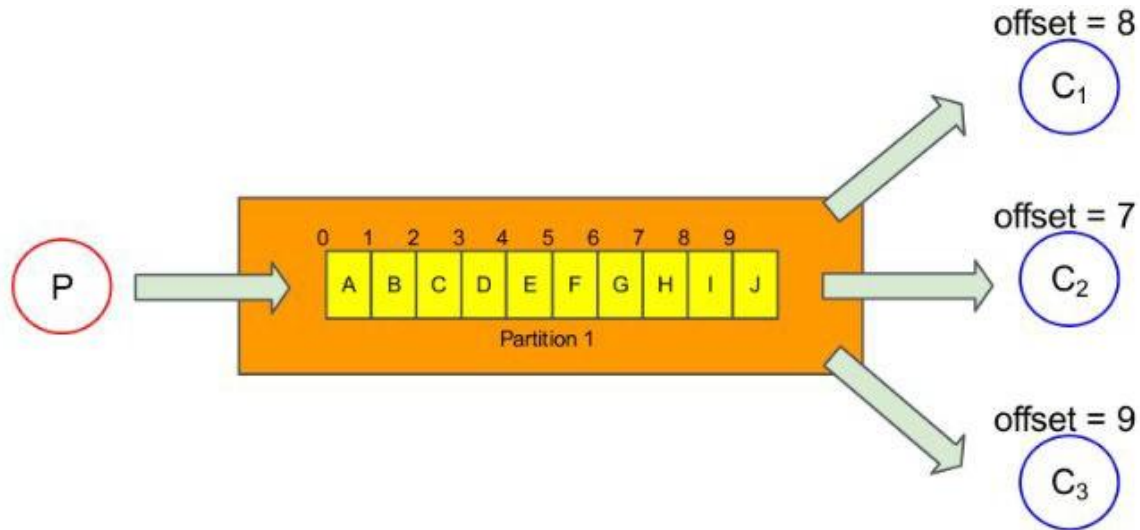
Conceptos de Apache Kafka

- Modelos de consumición
 - 1 partición, 1 consumidor ==> cola



Conceptos de Apache Kafka

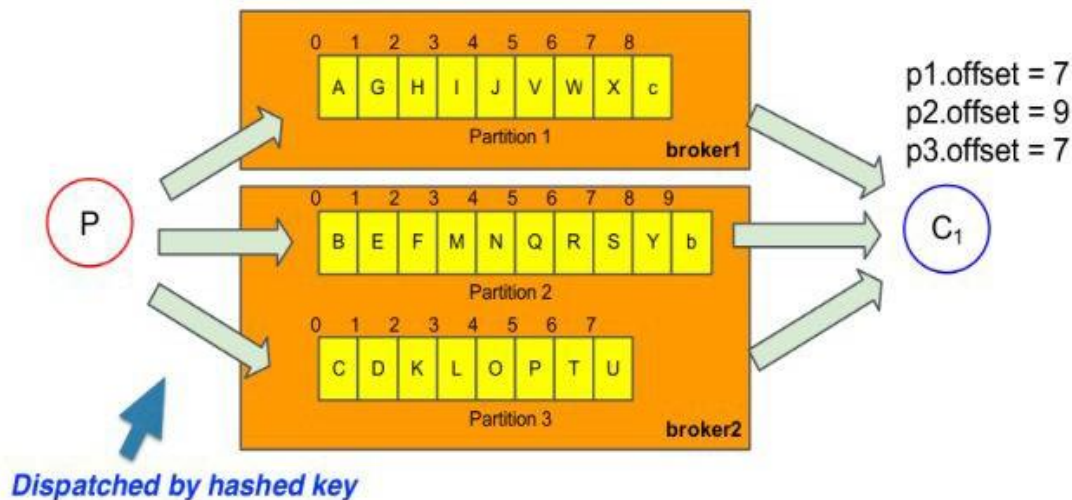
- Modelos de consumición
 - 1 partición, n consumidores ==> pub/sub



Each Consumer keep its own offset.

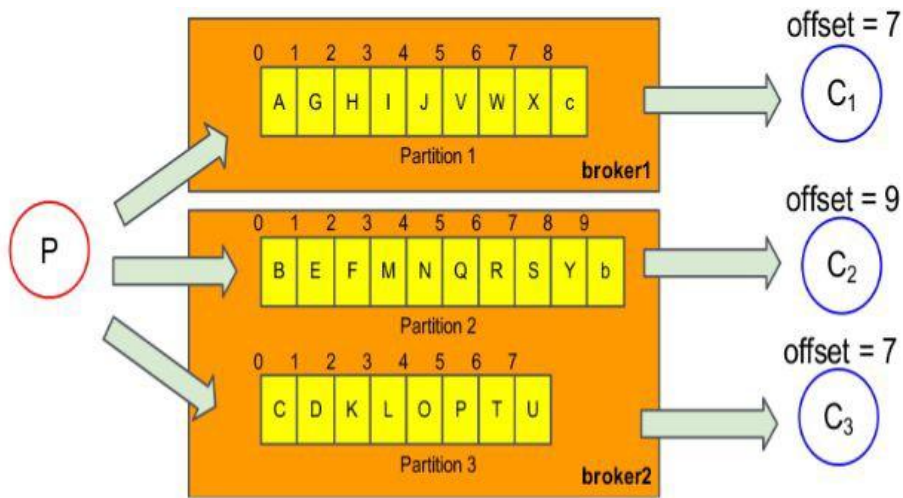
Conceptos de Apache Kafka

- Modelos de consumición
 - N particiones, 1 consumidor



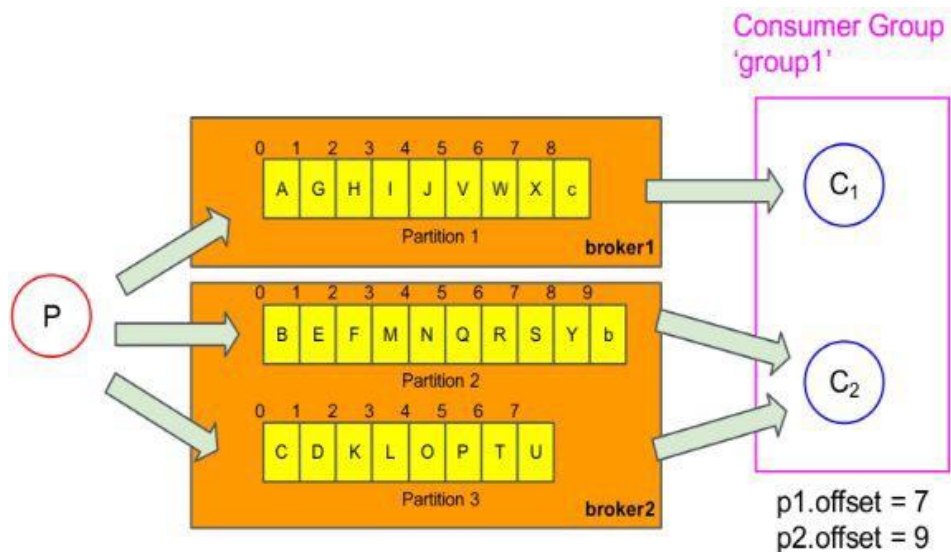
Conceptos de Apache Kafka

- Modelos de consumición
 - N particiones, n° consumidores



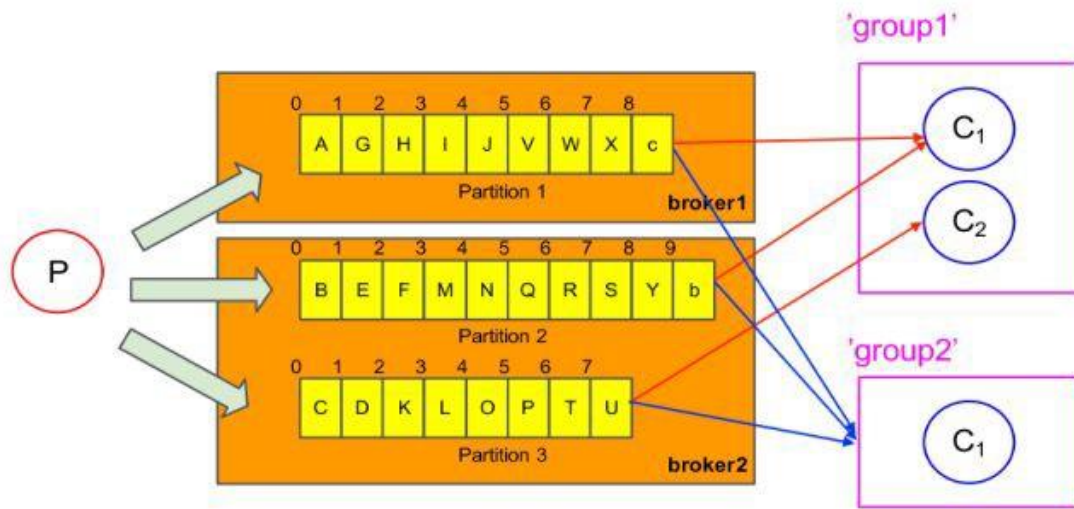
Conceptos de Apache Kafka

- Consumer groups
 - N particiones, 1 grupo de consumidores



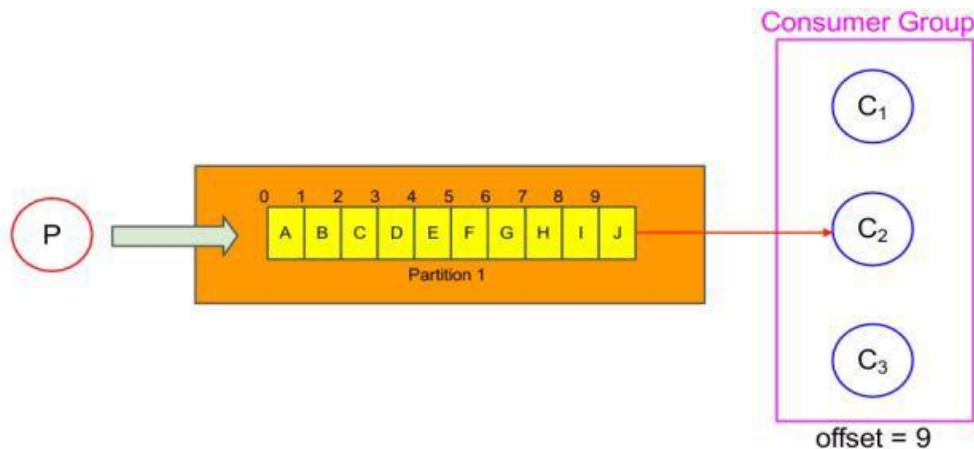
Conceptos de Apache Kafka

- Consumer groups
 - N particiones, 2 grupos de consumidores



Conceptos de Apache Kafka

- Consumer groups
 - 1 particiones, n consumidores en el mismo grupo



Partition to Consumer is Many to One relation (In One Consumer Group)

Orden de mensajes

- Los 2 mayores retos en una arquitectura de sistemas distribuidos son:
 - Exactly-once delivery
 - Entrega con orden de mensajes garantizado



Mathias Verraes
@mathiasverraes

Seguir



There are only two hard problems in distributed systems: 2. Exactly-once delivery
1. Guaranteed order of messages 2. Exactly-once delivery

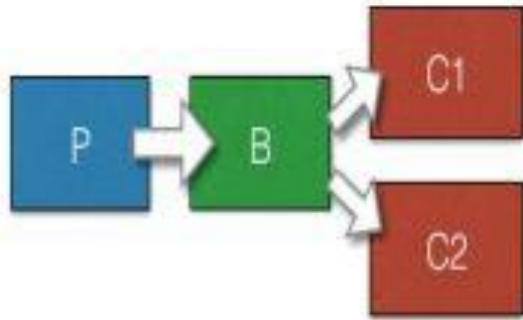
11:40 - 14 ago. 2015

7.190 Retweets 5.593 Me gusta

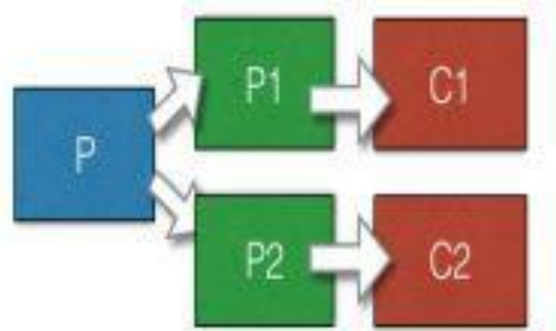


Orden de mensajes

- Sistema de mensajería tradicional garantizan el orden de entrega
- Kafka garantiza el orden de procesamiento dentro de la misma partición



Traditional MQ



Kafka

Semántica de entrega

- At most once:
 - Los mensajes se pueden perder pero nunca ser reenviados
- At least once:
 - Los mensajes nunca se pierden, pero podrían llegar varias veces
- Exactly once:
 - two-phase commit
 - at least once + idempotencia
 - at least once + deduplicación

Semántica de entrega

- At most once
 - Envío asíncrono
- At least once
 - Envío síncrono (contador de intentos)
- Exactly once
 - Envío asíncrono o síncrono



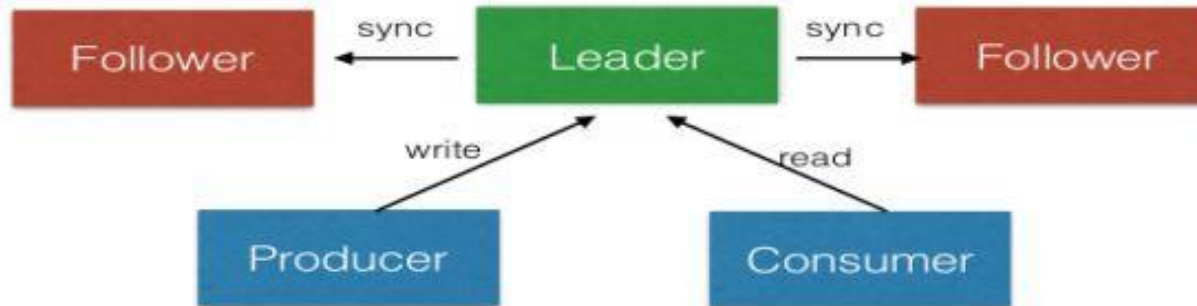
Semántica de entrega

- At most once
 - guardar offset antes de procesar los mensajes
- At least once
 - guardar offset después de procesar los mensajes
- Exactly once
 - Offset como identificador único en el consumidor



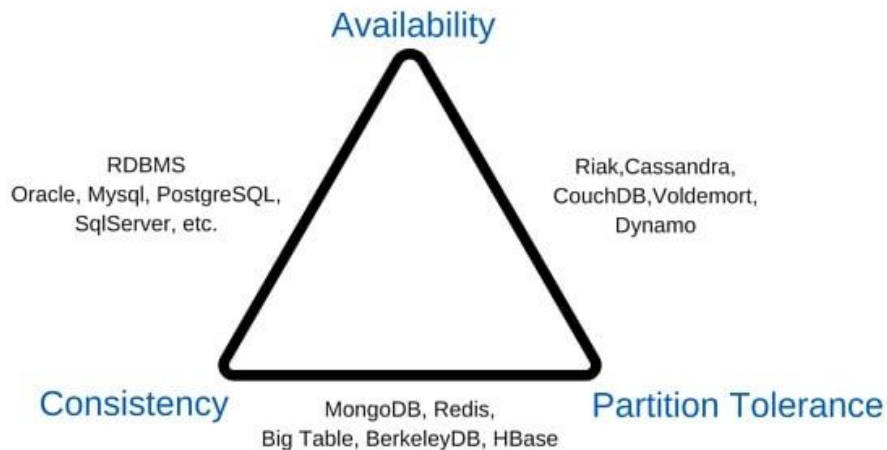
Replicación

- Replicación a nivel de partición
- Cada partición tiene un leader y 0 o más particiones
- Lecturas y escrituras a través del leader
- No aumento de throughput, solo backup
- In-sync replicas

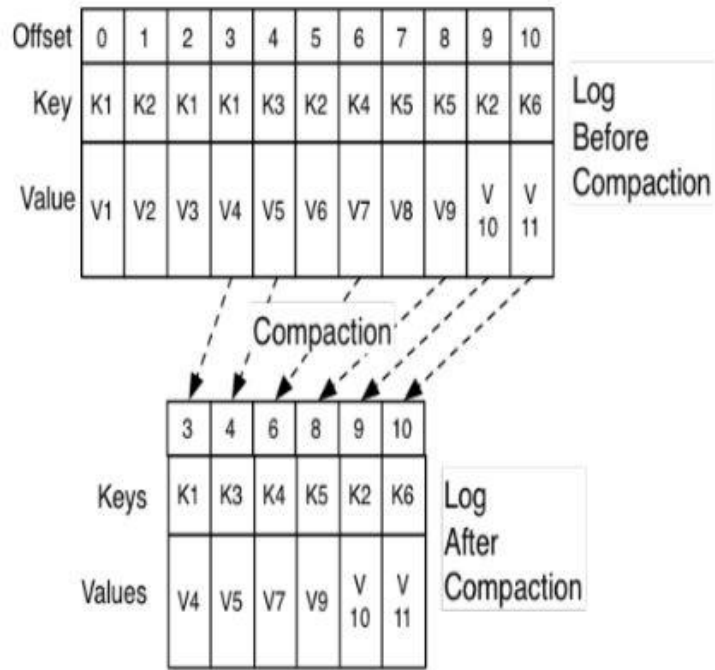


Replicación

- Garantía de no perder datos
 - Al menos una réplica sincronizada ok
- Disponibilidad vs consistencia



Compactación de log



Tolerancia a fallos

- ZK: mitad+1 para dar servicio (Quorum)
- Kafka: depende de la configuración cada tópico
 - número de réplicas
 - distribución de las réplicas

Escalabilidad

- Kafka soporta escalabilidad vertical y horizontal
 - Verticalmente
 - más disco permite aumentar el tamaño de las particiones
 - Horizontalmente
 - más brokers permiten manejar más particiones (operación manual)
 - más particiones permiten mayor throughput de consumición

Casos de uso de Apache Kafka

- LinkedIn: activity streams, operational metrics, data bus
- 400 nodes, 18k topics, 220B msg/day (peak 3.2M msg/s), May 2014
- Netflix: real-time monitoring and event processing
- Twitter: as part of their Storm real-time data pipelines
- Spotify: log delivery (from 4h down to 10s), Hadoop
- Loggly: log collection and processing
- Mozilla: telemetry data
- Airbnb, Cisco, Uber...

Errores comunes

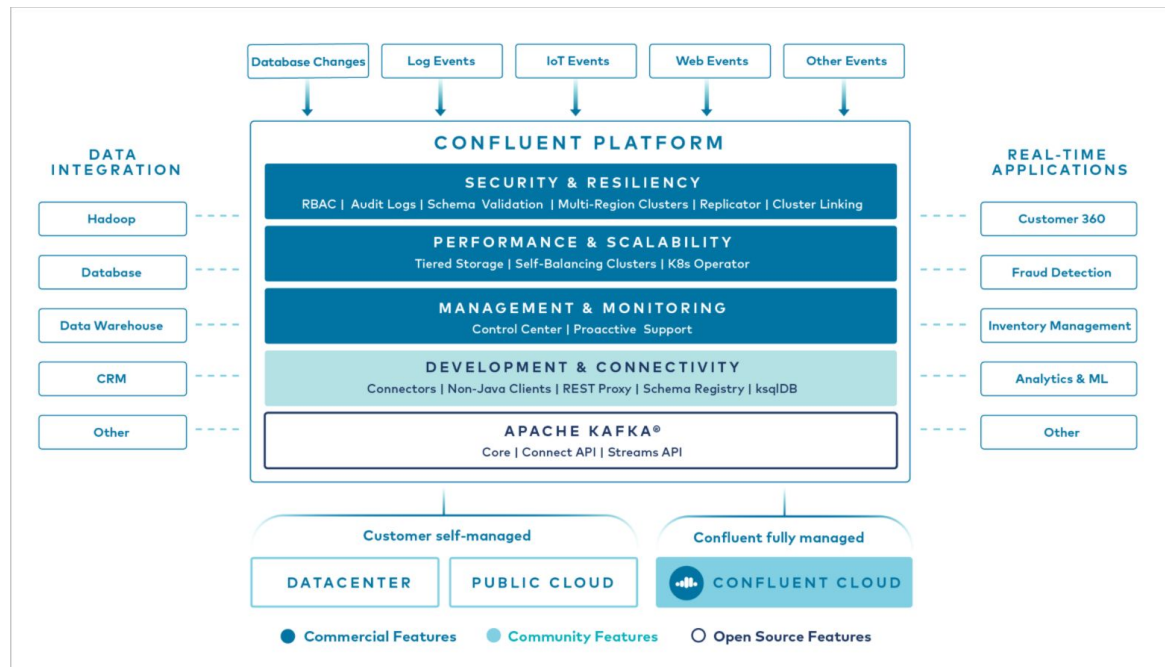
- Asumir que se va a consumir a la misma velocidad que se va a producir:
 - Usar particiones antes de romper en tópicos
- Asumir que el retraso en todas las particiones va a ser uniforme:
 - Utilizar claves para producir de tal manera que todo lo relacionado temporalmente quede dentro de la misma partición

Errores comunes

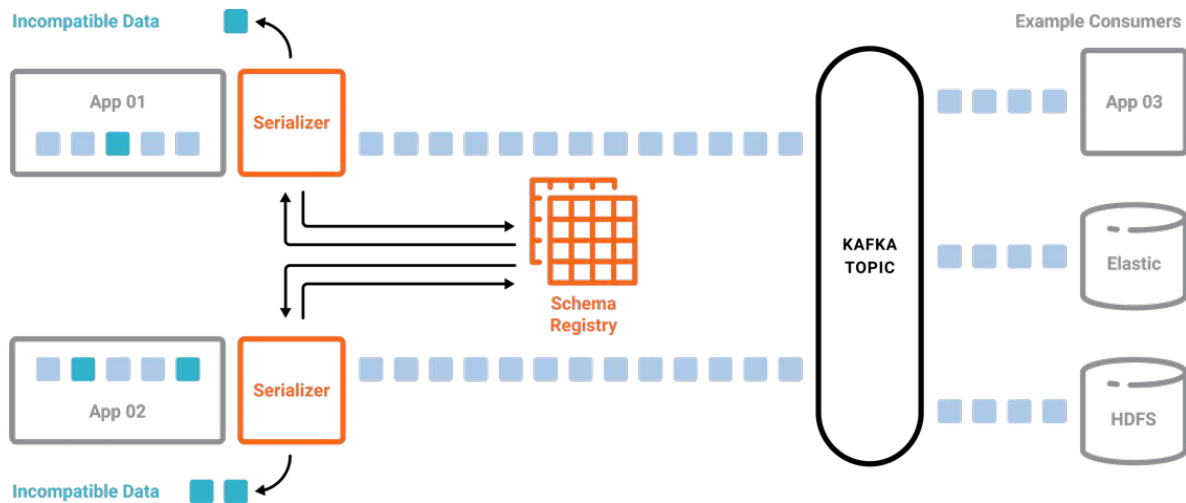
- Topic con líder -1 : el líder del topic se ha caído y no ha tomado ninguna replica su control
 - chequear comunicación con ZK
 - chequear controller y reiniciar 1
- Datos corruptos en ZK:
 - `java.lang.NumberFormatException: For input string: "consumers" at java.lang.NumberFormatException.forInputString(NumberFormatException.java:65)`
 - Debemos de conectarnos a ZK con CLI y chequear la integridad de los datos

Herramientas de Apache Kafka

Ecosistema



Schema Registry



Schema Registry

- Repositorio de esquemas Avro
- Backend en tópicos de Kafka
- Esquemas disponibles para ser usados como clave o como valor de los mensajes y ser asociados a un tópico
- Aplicaciones cliente pueden registrar sus propios esquemas (desaconsejado en producción)
- Soporta evolución de los esquemas
- Garantiza que la evolución de esquema respeta el contrato productor-consumidor

Schema Registry

• V1

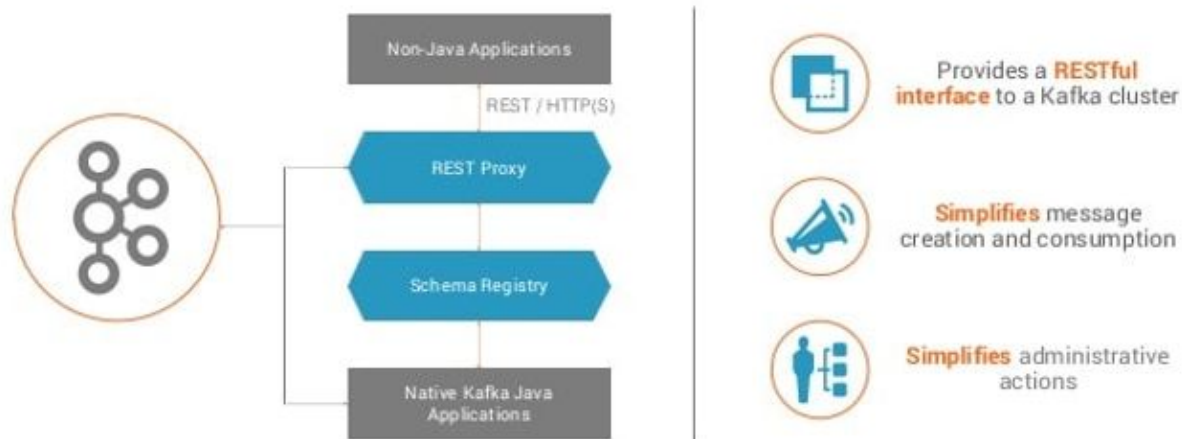
```
{ "namespace": "io.confluent.examples.clients.basicavro",  
  "type": "record",  
  "name": "Payment",  
  "fields": [  
    { "name": "id", "type": "string" },  
    { "name": "amount", "type": "double" }  
  ]  
}
```

• V2

```
{ "namespace": "io.confluent.examples.clients.basicavro",  
  "type": "record",  
  "name": "Payment",  
  "fields": [  
    { "name": "id", "type": "string" },  
    { "name": "amount", "type": "double" },  
    { "name": "region", "type": "string", "default": "" }  
  ]  
}
```

Rest Proxy

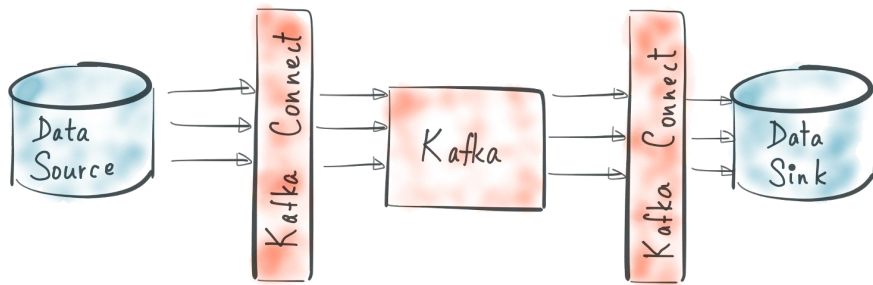
Confluent REST Proxy



Rest Proxy

- Cliente http con interfaz RESTful
 - Producción de mensajes
 - Consumición
 - Soportados consumer groups
 - Consulta de metadata del cluster
 - Tareas administrativas todavía no soportadas

Kafka Connect



Kafka Connect

- Kafka Connect es un componente que permite leer y escribir desde/en diversas fuentes de datos, tanto de streaming como de batch



Kafka Connect

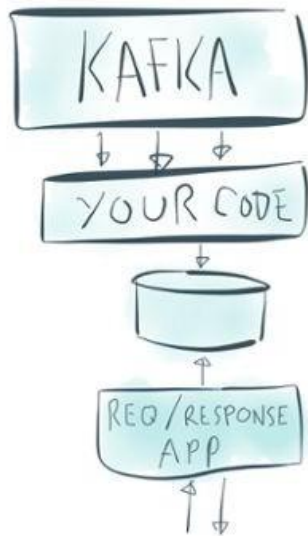
- Framework para conectar Kafka con sistemas externos
- Basado en conectores:
 - source connector (hacia Kafka)
 - sink connector (desde Kafka)
- Puede funcionar como un proceso standalone o como servicio distribuido (YARN, Mesos, Kubernetes...), escalable y tolerante a fallos
- Existen conectores soportados o podemos desarrollar nuestro propio conector

Kafka Connect

- Connectores soportados:
 - FileStream
 - HDFS
 - S3
 - Google Cloud Storage
 - JDBC
 - Elasticsearch
 - Cassandra
 - JMS
 - Active MQ IBM MQ

Kafka Streams

WITH KAFKA STREAMS



Kafka Streams

- Kafka Streams es un componente que permite realizar ETLs desde un topic a otro



Ejemplo:

Suma de ventas por minuto

Kafka Streams

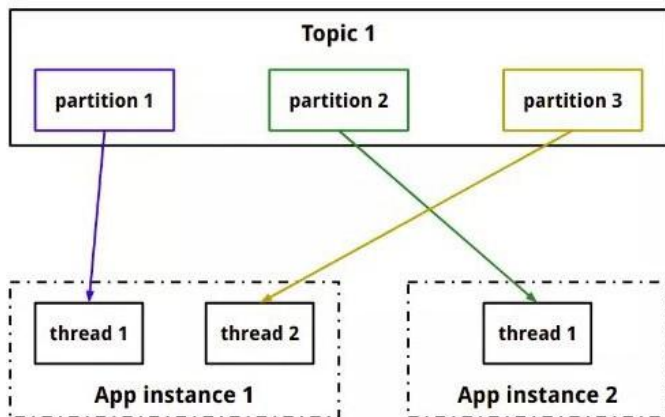
- Alternativa sencilla a otras soluciones de procesamiento en streaming:
 - Open source: spark, storm, flink...
 - Propietarias: AWS Kinesis, Google DataFlow
- Principal ventaja
 - Es una librería
- Desventaja
 - Kafka como única fuente de datos

Kafka Streams

- Características:
 - Procesado de eventos de 1 en 1 (con latencias de milisegundos)
 - Operaciones con estado: agregaciones, joins, ventanas...
 - Procesamiento distribuido, tolerante a fallos
 - Fail fast
 - Capacidades de reprocesado
 - Rolling deployments sin downtime
 - API de 2 niveles

Kafka Streams

- Paralelismo y escalado:
 - Basado en consumer groups
 - Grado máximo
 - Número de particiones
 - Escalado horizontal, vertical o combinado



Kafka Streams

- Tolerancia a fallos
 - Basada en los offsets de Kafka
 - Si una tarea falla otra prosigue desde el mismo punto en el que se quedó
 - En función del error, esto puede suponer un problema (fail fast puede causarnos bucles infinitos de procesamiento)

Kafka Streams

- En aplicaciones event-driven no es necesario aportar mecanismos extras de backpressure
- Semántica de procesamiento
 - La misma que Kafka
 - En la mayoría de escenarios at least once es suficiente
 - En caso de necesitar deduplicación KVStore (in memory, persistent, windowed...)
- Especialmente recomendado para escenarios cloud/distribuidos

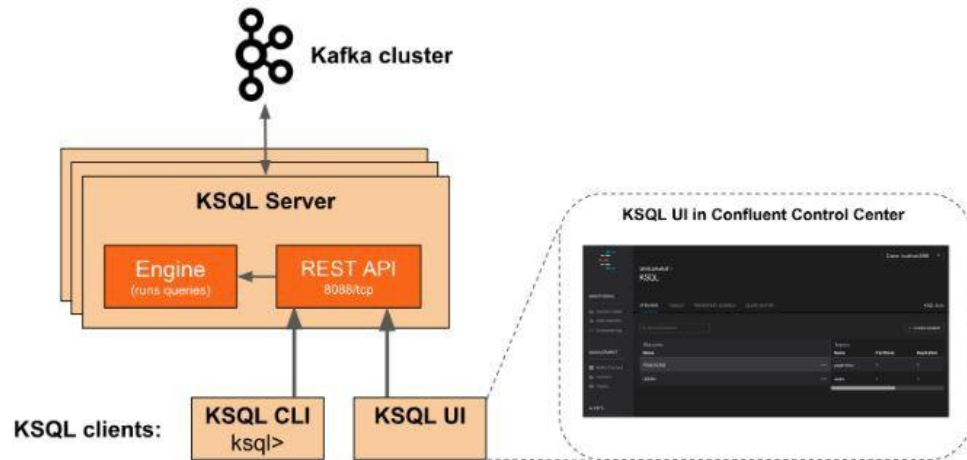
Kafka Streams

- API a 2 niveles:
 - DSL
 - Transformaciones sin estado; map, flatmap, filter, foreach, groupbykey...
 - Transformaciones con estado: reduce, agregaciones, ventanas...

Kafka Streams

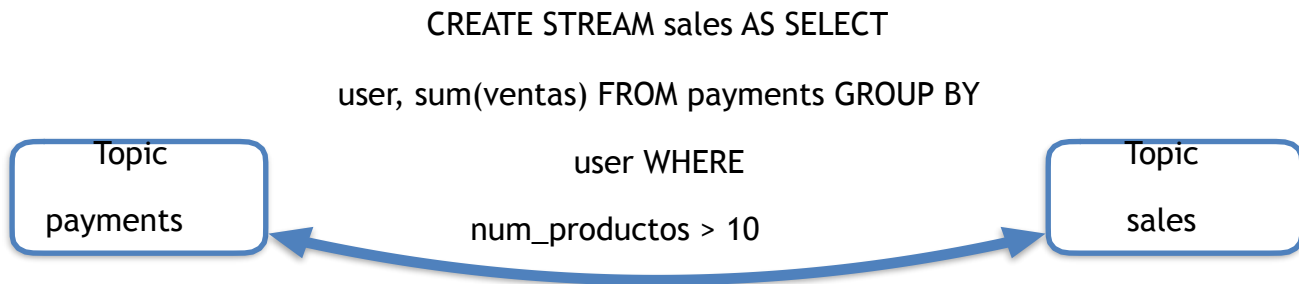
- API a 2 niveles:
 - Processor API:
 - Permite definir a bajo nivel operaciones con o sin estado (Ej: guardar cada evento en una bd externa)
 - Si la operación requiere de estado se usan state stores (Ej: deduplicaciones)
 - Se pueden definir 2 tipos de operaciones:
 - Processors: no devuelven nada al stream
 - Transformers: devuelven algo al stream

KSQL



KSQL

- KSQL es un componente que permite hacer lo mismo que Kafka Streams pero con sentencias SQL-like



KSQL vs Kafka Streams

```
val fraudulentPayments: KStream[String, Payment] = builder
    .stream[String, Payment]("payments-kafka-topic")
    .filter((_ ,payment) => payment.fraudProbability > 0.8)

fraudulentPayments.to("fraudulent-payments-topic")
```


¡Gracias!